

字节面试手撕 Top50 (ACM)

ACM常用处理函数

1.bufio

```
// 创建一个缓冲读取器, 从标准输入中读取数据
reader := bufio.NewReader(os.Stdin)

line1, _ := reader.ReadString('\n')

line2, _ := reader.ReadString('\n')

// line1 := "5 9\n"
// line2 := "1 2 3 4 5\n"
```

```
scanner := bufio.NewScanner(os.Stdin)

scanner.Scan()

line := scanner.Text()
```

2.os

3.strings

```
// strings.Fields 转为切片
desc := strings.Fields(line1)

numsStr := strings.Fields(line2)

// desc = []string{"5", "9"}
// numsStr = []string{"1", "2", "3", "4", "5"}
```

```
// strings.Join 将字符串切片, 拼接为字符串
// 只能为string, 如果是int, 要先strconv.Itoa()
res := []string{"5", "4", "3", "2", "1"}
s := strings.Join(res, " ")
fmt.Println(s)
```

```
line, _ := reader.ReadString('\n')
line = strings.TrimSpace(line) // 去除换行符
matrix[i] = make([]byte, n)
for j := 0; j < n; j++ {
    matrix[i][j] = line[j]
}
```

4. strconv

```
// strconv.Atoi → ASCII to int
// strconv.Itoa → int to ASCII

num, _ := strconv.Atoi(s)

s := strconv.Itoa(123)
```

5. fmt

```
var n int
fmt.Scan(&n)
```

```
fmt.Println(res[0], res[1])

fmt.Println(strings.Join(res, " "))
```

```
var numsLen, target int

fmt.Fscan(os.Stdin, &numsLen, &target)

nums := make([]int, numsLen)
for i := 0; i < numsLen; i++ {
    fmt.Fscan(os.Stdin, &nums[i])
}
```

6. sort

```
// 把 nums 这个整型切片按从小到大排序（升序）
sort.Ints(nums)
```

7. rand

```
import (
    "math/rand"
)

rand.Seed(time.Now().UnixNano())

pivotIndex := left + rand.Intn(right - left + 1)
```

哈希 (1)

1 两数之和 ✓

```
package main

import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)

func twoSum(nums []int, target int) []int {
    hashMap := make(map[int]int, 0)

    for k, num := range nums {
        sub := target - num
        if index, ok := hashMap[sub]; ok {
            return []int {index, k}
        }

        hashMap[num] = k
    }

    // 一定要注意这个边界条件
    return []int{-1, -1}
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    // 读取n和target
    // 这里一定要用 '\n', 不能用 "\n", 否则会报错
    line1, _ := reader.ReadString('\n')
    parts := strings.Fields(line1)
    n, _ := strconv.Atoi(parts[0])
    target, _ := strconv.Atoi(parts[1])

    // 读取数组
```

```

line2, _ := reader.ReadString('\n')
numsStr := strings.Fields(line2)
nums := make([]int, n)
for i := 0; i < n; i++ {
    nums[i], _ = strconv.Atoi(numsStr[i])
}

// 调用twoSum函数并输出结果
res := twoSum(nums, target)
fmt.Println(res[0], res[1])
}

```

双指针（2）

1 三数之和

```

package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
    "sort"
)

func threeSum(nums []int) [][]int {
    res := make([][]int, 0)

    sort.Ints(nums)
    n := len(nums)

    for i := 0; i < n - 2; i++ {
        // 如果排序后的第一个值都大于0，那么直接返回结束
        if nums[i] > 0 {
            break
        }

        // 跳过重复值
        if i > 0 && nums[i] == nums[i-1] {
            continue
        }

        left, right := i + 1, n - 1

        // 双指针
        for left < right {

```

```

        sum := nums[i] + nums[left] + nums[right]

        if sum == 0 {
            res = append(res, []int {nums[i], nums[left], nums[right]})

            for left < right && nums[left] == nums[left+1] {
                left++
            }

            for left < right && nums[right] == nums[right-1] {
                right--
            }

            left++
            right--
        } else if sum < 0 {
            left++
        } else {
            right--
        }
    }
}

return res
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    line2, _ := reader.ReadString('\n')

    desc := strings.Fields(line1)
    numsStr := strings.Fields(line2)

    numsLen, _ := strconv.Atoi(desc[0])

    nums := make([]int, numsLen)
    for k, v := range numsStr {
        val, _ := strconv.Atoi(numsStr[k])
        nums[k] = val
    }

    res := threeSum(nums)

    for _, numGroup := range res {
        fmt.Println(numGroup[0], numGroup[1], numGroup[2])
    }
}

```

```

package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
)

func obtainRain(height []int) int {
    if len(height) == 0 {
        return 0
    }

    var rain int

    left, right := 0, len(height)-1
    leftMax, rightMax := height[left], height[right]

    for left < right {
        if height[left] < height[right] {
            if height[left] > leftMax {
                leftMax = height[left]
            } else {
                rain += leftMax - height[left]
            }
            left++
        } else {
            if height[right] > rightMax {
                rightMax = height[right]
            } else {
                rain += rightMax - height[right]
            }
            right--
        }
    }

    return rain
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    line2, _ := reader.ReadString('\n')

    desc := strings.Fields(line1)
    heightLen, _ := strconv.Atoi(desc[0])

    numsStr := strings.Fields(line2)
    nums := make([]int, heightLen)

```

```

    for k, v := range numsStr {
        val, _ := strconv.Atoi(v)
        nums[k] = val
    }

    rain := obtainRain(nums)

    fmt.Println(rain)
}

```

滑动窗口（2）

3 无重复字符的最长子串

```

package main

import (
    "bufio"
    "os"
    "fmt"
)

func longestSubstring(str string) int {
    lastSeen := make(map[rune]int, len(str))
    runes := []rune(str)

    maxLen := 0
    start := 0

    for i := 0; i < len(runes); i++ {
        // 当前 ch 之前已经出现过了, 并且位置是在起点的前面
        // 更新滑动窗口的值
        if pos, exists := lastSeen[runes[i]]; exists && pos >= start {
            start = pos + 1
        }

        // 更新滑动窗口最大值
        currentLen := i - start + 1
        if currentLen > maxLen {
            maxLen = currentLen
        }

        // 更新 lastSeen map
        lastSeen[runes[i]] = i
    }

    return maxLen
}

func main() {

```

```

scanner := bufio.NewScanner(os.Stdin)

scanner.Scan()
line := scanner.Text()

// line = "abcabcbb"
res := longestSubstring(line)

fmt.Println(res)
}

```

3 长度最小的子数组

```

...
right=0 → sum=2
right=1 → sum=5
right=2 → sum=6
right=3 → sum=8 

窗口: [2,3,1,2] → 长度=4

开始收缩:
去掉2 → sum=6 ✕

right=4 → sum=10 
窗口: [3,1,2,4] → 长度=4

收缩:
去掉3 → sum=7  → 长度=3
去掉1 → sum=6 ✕

right=5 → sum=9 
窗口: [2,4,3]

收缩:
去掉2 → sum=7  → 长度=2 ☆
...

```

```

package main

import (
    "fmt"
)

func findShortestNumsLen(nums []int, target int) int {
    left := 0
    sum := 0
    minLen := len(nums) + 1

    for right := 0; right < len(nums); right++ {
        sum += nums[right]

        for sum >= target {

```



```

        if right - left + 1 < minLen {
            minLen = right - left + 1
        }
        sum -= nums[left]
        left++
    }
}

if minLen == len(nums) + 1 {
    return 0
}

return minLen
}

func main() {
    var numsLen, target int
    fmt.Scan(&numsLen, &target)

    nums := make([]int, numsLen)
    for i := 0; i < numsLen; i++ {
        fmt.Scan(&nums[i])
    }

    res := findShortestNumsLen(nums, target)

    fmt.Println(res)
}

```

字符串 (3)

3 滑动窗口最大值

```

package main

import (
    "fmt"
    "os"
)

func maxSlidewindow(nums []int, k int) []int {
    // 初始化一个单调递减的双端队列
    // 这个队列里面，存储的是索引，而不是数组的值
    res := make([]int, len(nums)-k+1)
    deque := make([]int, 0)

    for i := 0; i < len(nums); i++ {
        // 队首元素出队
        if len(deque) > 0 && deque[0] < i-k+1 {
            deque = deque[1:]
        }
    }
}

```

```

        // 队尾元素出队
        for len(deque) > 0 && nums[deque[len(deque)-1]] < nums[i] {
            deque = deque[:len(deque)-1]
        }

        // 当前索引入队
        deque = append(deque, i)

        // 将当前滑动窗口索引值, 对应的nums值, 添加到res
        if i >= k-1 {
            res[i-k+1] = nums[deque[0]]
        }
    }

    return res
}

func main() {
    var numsLen, k int
    fmt.Fscan(os.Stdin, &numsLen, &k)

    nums := make([]int, numsLen)
    for i := 0; i < numsLen; i++ {
        fmt.Fscan(os.Stdin, &nums[i])
    }

    res := maxSlidewindow(nums, k)

    for i := 0; i < len(res); i++ {
        fmt.Print(res[i])
        if i != len(res)-1 {
            fmt.Print(" ")
        }
    }
}

```

比较版本号

字符串相加

普通数组 (3)

合并两个有序数组

最大子数组和

合并区间

矩阵 (1)

1 螺旋矩阵 ✓

```
package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
)

func spinMatrix(matrix [][]int) []int {
    if len(matrix) == 0 || len(matrix[0]) == 0 {
        return []int{}
    }

    m, n := len(matrix), len(matrix[0])

    left, right := 0, n-1
    top, bottom := 0, m-1

    res := make([]int, 0, m*n)

    for left <= right && top <= bottom {
        // 左 - 右
        for col := left; col <= right; col++ {
            res = append(res, matrix[top][col])
        }
        top++

        // 上 - 下
        for row := top; row <= bottom; row++ {
            res = append(res, matrix[row][right])
        }
        right--

        // 右 - 左
        if top <= bottom {
            for col := right; col >= left; col-- {
                res = append(res, matrix[bottom][col])
            }
            bottom--
        }

        // 左 - 上
        if left <= right {
            for row := bottom; row >= top; row-- {
                res = append(res, matrix[row][left])
            }
        }
    }
}
```

```

        left++
    }
}

return res
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    desc := strings.Fields(line1)
    m, _ := strconv.Atoi(desc[0])
    n, _ := strconv.Atoi(desc[1])

    matrix := make([][]int, m)
    for i := 0; i < m; i++ {
        line, _ := reader.ReadString('\n')
        numsStr := strings.Fields(line)
        nums := make([]int, n)
        for k, v := range numsStr {
            val, _ := strconv.Atoi(v)
            nums[k] = val
        }
        matrix[i] = nums
    }

    res := spinMatrix(matrix)

    resStr := make([]string, 0)
    for k, v := range res {
        val := strconv.Itoa(v)
        resStr = append(resStr, val)
    }

    fmt.Println(strings.Join(resStr, " "))
}

```

链表 (10)

1 反转链表

```

package main

import (
    "fmt"
    "bufio"
    "os"

```

```

    "strings"
    "strconv"
)

type ListNode struct {
    val int
    Next *ListNode
}

func reverseList(head *ListNode) *ListNode {
    if head == nil {
        return nil
    }

    var prev *ListNode
    cur := head

    for cur != nil {
        next := cur.Next
        cur.Next = prev
        prev = cur
        cur = next
    }

    return prev
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    // 读取链表长度n 和 节点值
    line1, _ := reader.ReadString('\n')
    line2, _ := reader.ReadString('\n')

    desc := strings.Fields(line1)
    numsStr := strings.Fields(line2)

    n, _ := strconv.Atoi(desc[0])
    if n <= 0 {
        return
    }

    // 构造链表
    dummy := &ListNode{}
    cur := dummy
    for i := 0; i < n; i++ {
        val, _ := strconv.Atoi(numsStr[i])
        cur.Next = &ListNode{Val: val}
        cur = cur.Next
    }

    // 反转链表并返回
    reversedHeadNode := reverseList(dummy.Next)

```

```
cur = reversedHeadNode

res := make([]string, 0)
for cur != nil {
    res = append(res, strconv.Itoa(cur.Val))
    cur = cur.Next
}
fmt.Println(strings.Join(res, " "))
}
```

删除链表的倒数第 N 个节点

K 个一组翻转链表

合并 K 个升序链表

合并两个有序链表

排序链表

环形链表

两数相加

反转链表2

LRU缓存

二叉树（6）

二叉树的最近公共祖先

二叉树的锯齿形层序遍历

1 层序遍历二叉树

给定一个整数数组 `nums`，其中 `nums[i]` 表示二叉树节点的值。按照以下规则构建**二叉树**：

规则1： **根节点**： `nums[0]`

规则2： **左子节点**： `nums[i]` 的左子节点是 `nums[2i+1]`

规则3： **右子节点**： `nums[i]` 的右子节点是 `nums[2i+2]`

规则4： `-1` 表示 **空节点**，在构建树时跳过该节点。

```

package main

import (
    "fmt"
    "bufio"
    "os"
    "strings"
    "strconv"
)

type TreeNode struct {
    val int
    Left *TreeNode
    Right *TreeNode
}

func levelOrder(root *TreeNode) [][]int {
    if root == nil {
        return nil
    }

    res := make([][]int, 0)
    queue := []*TreeNode{root}

    for len(queue) > 0 {
        layerSize := len(queue)
        layer := []int {}

        for i := 0; i < layerSize; i++ {
            node := queue[0]
            layer = append(layer, node.val)
            queue = queue[1:]

            if node.Left != nil {
                queue = append(queue, node.Left)
            }

            if node.Right != nil {
                queue = append(queue, node.Right)
            }
        }

        res = append(res, layer)
    }

    return res
}

func buildBinaryTree(nums []int) *TreeNode {
    if len(nums) == 0 || nums[0] == -1 {
        return nil
    }

```

```

nodes := make([]*TreeNode, len(nums))
for i, val := range(nums) {
    if val != -1 {
        nodes[i] = &TreeNode{Val: val}
    }
}

for i := 0; i < len(nums); i++ {
    if nodes[i] == nil {
        continue
    }

    leftIndex := 2*i + 1
    rightIndex := 2*i + 2

    if leftIndex < len(nums) {
        nodes[i].Left = nodes[leftIndex]
    }

    if rightIndex < len(nums) {
        nodes[i].Right = nodes[rightIndex]
    }
}

return nodes[0]
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    // 读取数据
    line1, _ := reader.ReadString('\n')
    numsStr := strings.Fields(line1)

    nums := make([]int, 0)
    for _, val := range numsStr {
        num, _ := strconv.Atoi(val)
        nums = append(nums, num)
    }

    // 构造二叉树
    root := buildBinaryTree(nums)

    // 层序遍历
    res := levelOrder(root)

    // 最终输出
    for _, layer := range(res) {
        for _, v := range(layer) {
            fmt.Println(v)
        }
    }
}

```


1 二叉树的右视图 ✓

```
package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
)

type TreeNode struct {
    val int
    Left *TreeNode
    Right *TreeNode
}

func rightTree(root *TreeNode) []int {
    if root == nil {
        return []int {}
    }

    res := make([]int, 0)
    queue := []*TreeNode {root}

    for len(queue) > 0 {
        layerSize := len(queue)
        for i := 0; i < layerSize; i++ {
            node := queue[0]
            queue = queue[1:]

            if i == layerSize - 1 {
                res = append(res, node.val)
            }

            if node.Left != nil {
                queue = append(queue, node.Left)
            }

            if node.Right != nil {
                queue = append(queue, node.Right)
            }
        }
    }

    return res
}

// func buildBinaryTree(numsStr []string, treeLen int) *TreeNode {
//     if treeLen <= 0 || numsStr[0] == "null" {
//         return nil
//     }
// }
```

```

//      nodes := make([]*TreeNode, treeLen)
//      for k, v := range numsStr {
//          if v != "null" {
//              val, _ := strconv.Atoi(v)
//              nodes[k] = &TreeNode{Val: val}
//          }
//      }

//      for i := 0; i < treeLen; i++ {
//          if numsStr[i] == "null" {
//              continue
//          }

//          leftIndex := 2*i + 1
//          rightIndex := 2*i + 2

//          if leftIndex < treeLen {
//              nodes[i].Left = nodes[leftIndex]
//          }

//          if rightIndex < treeLen {
//              nodes[i].Right = nodes[rightIndex]
//          }
//      }

//      return nodes[0]
//  }

func buildBinaryTree(numsStr []string, treeLen int) *TreeNode {
    if treeLen == 0 || numsStr[0] == "null" {
        return nil
    }

    // 创建根节点
    val, _ := strconv.Atoi(numsStr[0])
    root := &TreeNode{Val: val}

    // 使用队列进行层序构建
    queue := []*TreeNode{root}
    index := 1

    for len(queue) > 0 && index < len(numsStr) {
        node := queue[0]
        queue = queue[1:]

        // 构建左子节点
        if index < len(numsStr) && nodes[index] != "null" {
            val, _ := strconv.Atoi(numsStr[index])
            node.Left = &TreeNode{Val: val}
            queue = append(queue, node.Left)
        }
        index++
    }
}

```

```

        // 构建右子节点
        if index < len(numsStr) && numsStr[index] != "null" {
            val, _ := strconv.Atoi(numsStr[index])
            node.Right = &TreeNode{Val: val}
            queue = append(queue, node.Right)
        }
        index++
    }

    return root
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    line2, _ := reader.ReadString('\n')

    desc := strings.Fields(line1)
    numsStr := strings.Fields(line2)

    treeLen, _ := strconv.Atoi(desc[0])

    root := buildBinaryTree(numsStr, treeLen)

    res := rightTree(root)
    resStr := make([]string, len(res))
    for k, v := range res {
        resStr[k] = strconv.Itoa(v)
    }

    fmt.Println(strings.Join(resStr, " "))
}

```

对称二叉树

另一棵树的子树

搜索与图论（2）

2 岛屿数量

```
package main
```

```
import (
```

```

    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
)

func landNums(matrix [][]byte) int {
    if len(matrix) == 0 || len(matrix[0]) == 0 {
        return 0
    }

    m, n := len(matrix), len(matrix[0])
    count := 0

    var dfs func(int, int)
    dfs = func(i, j int) {
        if i < 0 || i >= m || j < 0 || j >= n || matrix[i][j] != '1' {
            return
        }

        matrix[i][j] = '0'

        dfs(i-1, j)
        dfs(i+1, j)
        dfs(i, j-1)
        dfs(i, j+1)
    }

    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if matrix[i][j] == '1' {
                count++
                dfs(i, j)
            }
        }
    }
    return count
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    desc := strings.Fields(line1)
    m, _ := strconv.Atoi(desc[0])
    n, _ := strconv.Atoi(desc[1])

    matrix := make([][]byte, m)
    for i := 0; i < m; i++ {
        line, _ := reader.ReadString('\n')
        line := strings.TrimSpace(line)
        matrix[i] = make([]byte, n)
    }
}

```

```

        for j := 0; j < n; j++ {
            matrix[i][j] = line[j]
        }
    }

    count := landNums(matrix)

    fmt.Println(count)
}

```

全排列

栈与队列 (3)

2 数组中的第 K 个最大元素 ✓

```

package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
    "math/rand"
    "time"
)

func findKthLargest(nums []int, k int) int {
    rand.Seed(time.Now().UnixNano())

    return quickSelect(nums, 0, len(nums)-1, k)
}

func quickSelect(nums []int, left, right, k int) int {
    if left == right {
        return nums[left]
    }

    pivotIndex := left + rand.Intn(right-left+1)
    nums[pivotIndex], nums[right] = nums[right], nums[pivotIndex]

    newPivotIndex := partition(nums, left, right)

    if newPivotIndex == k-1 {
        return nums[newPivotIndex]
    } else if newPivotIndex > k-1 {
        return quickSelect(nums, left, newPivotIndex-1, k)
    }
}

```

```

    } else {
        return quickSelect(nums, newPivotIndex+1, right, k)
    }
}

func partition(nums []int, left, right int) int {
    pivot := nums[right]

    i := left
    for j := left; j < right; j++ {
        if nums[j] > pivot {
            nums[i], nums[j] = nums[j], nums[i]
            i++
        }
    }

    nums[i], nums[right] = nums[right], nums[i]
    return i
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    line2, _ := reader.ReadString('\n')

    desc := strings.Fields(line1)
    numsLen, _ := strconv.Atoi(desc[0])
    k, _ := strconv.Atoi(desc[1])

    numsStr := strings.Fields(line2)
    nums := make([]int, numsLen)
    for key, v := range numsStr {
        val, _ := strconv.Atoi(v)
        nums[key] = val
    }

    res := findKthLargest(nums, k)

    fmt.Println(res)
}

```

有效的括号

用栈实现队列

贪心 (2)

2 买卖股票的最佳时机 ✓

```
package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
)

func bestProfit(nums []int) int {
    if len(nums) == 0 {
        return 0
    }

    maxProfit := 0
    minPrice := nums[0]

    for i := 0; i < len(nums); i++ {
        if nums[i] < minPrice {
            minPrice = nums[i]
        } else {
            if nums[i] - minPrice > maxProfit {
                maxProfit = nums[i] - minPrice
            }
        }
    }

    return maxProfit
}

func main() {
    scanner := bufio.NewScanner(os.Stdin)

    scanner.Scan()
    line := scanner.Text()
    numsStr := strings.Fields(line)

    nums := make([]int, len(numsStr))
    for k, v := range numsStr {
        val, _ := strconv.Atoi(v)
        nums[k] = val
    }

    maxPrice := bestProfit(nums)

    fmt.Println(maxPrice)
}
```

买卖股票的最佳时机2

动态规划（4）

2 爬楼梯1

```
package main

import (
    "fmt"
)

func climbStairs(n int) int {
    if n == 1 {
        return 1
    }

    if n == 2 {
        return 2
    }

    n1, n2 := 1, 2
    for i := 3; i <= n; i++ {
        n1, n2 = n2, n1 + n2
    }

    return n2
}

func main() {
    var n int
    fmt.Scan(&n)

    res := climbStairs(n)

    fmt.Println(res)
}
```

零钱兑换

最长递增子序列

最大为 N 的数字组合

多维动态规划（4）

最长回文子串

编辑距离

最长公共子序列

最长重复子数组

技巧（1）

2 寻找重复数

```
package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
)

func findDuplicate(nums []int) int {
    slow, fast := nums[0], nums[nums[0]]

    for slow != fast {
        slow = nums[slow]
        fast = nums[nums[fast]]
    }

    slow = 0
    for slow != fast {
        slow = nums[slow]
        fast = nums[fast]
    }

    return slow
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    line2, _ := reader.ReadString('\n')

    desc := strings.Fields(line1)
    numsLen, _ := strconv.Atoi(desc[0])
}
```

```

numsStr := strings.Fields(line2)
nums := make([]int, numsLen+1)
for k, v := range numsStr {
    val, _ := strconv.Atoi(v)
    nums[k] = val
}

res := findDuplicate(nums)

fmt.Println(res)
}

```

二分查找 (5)

3 在排序数组中查找元素的第一个和最后一个位置 ✓

```

package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
)

func searchRange(nums []int, target int) []int {
    leftIndex := searchBinary(nums, target)
    rightIndex := searchBinary(nums, target+1) - 1

    if leftIndex == len(nums) || nums[leftIndex] != target {
        return []int {-1, -1}
    }

    return []int {leftIndex, rightIndex}
}

func searchBinary(nums []int, target int) int {
    left, right := 0, len(nums)-1

    for left <= right {
        mid := (left + right) / 2

        if nums[mid] < target {
            left = mid + 1
        } else {
            right = mid - 1
        }
    }

    return left
}

```

```

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    line2, _ := reader.ReadString('\n')
    line3, _ := reader.ReadString('\n')

    desc := strings.Fields(line1)
    numsStr := strings.Fields(line2)

    numsLen, _ := strconv.Atoi(desc[0])

    nums := make([]int, numsLen)
    for k, v := range numsStr {
        val, _ := strconv.Atoi(v)
        nums[k] = val
    }

    targetStr := strings.Fields(line3)
    target, _ := strconv.Atoi(targetStr[0])

    res := searchRange(nums, target)

    fmt.Println(res[0], res[1])
}

```

3 搜索旋转排序数组

```

package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
)

func search(nums []int, target int) int {
    left, right := 0, len(nums)-1

    for left <= right {
        mid := (left + right) / 2

        if nums[mid] == target {
            return mid
        }

        if nums[left] <= nums[mid] {
            if target >= nums[left] && target < nums[mid] {
                right = mid - 1
            }
        }
    }
}

```

```

        } else {
            left = mid + 1
        }
    } else {
        if target > nums[mid] && target <= nums[right] {
            left = mid + 1
        } else {
            right = mid - 1
        }
    }
}

return -1
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    line2, _ := reader.ReadString('\n')
    line3, _ := reader.ReadString('\n')

    desc1 := strings.Fields(line1)
    numsStr := strings.Fields(line2)
    desc2 := strings.Fields(line3)

    numsLen, _ := strconv.Atoi(desc1[0])
    nums := make([]int, numsLen)
    for k, v := range numsStr {
        val, _ := strconv.Atoi(v)
        nums[k] = val
    }

    target, _ := strconv.Atoi(desc2[0])

    res := search(nums, target)

    fmt.Print(res)
}

```

3 x 的平方根 ✓

```

package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"

```

```

)

func sqrt(target int) int {
    if target < 2 {
        return target
    }

    left, right := 0, target-1
    for left <= right {
        mid := (left + right) / 2

        if mid * mid == target {
            return mid
        } else if mid * mid > target {
            right = mid - 1
        } else {
            left = mid + 1
        }
    }

    return right
}

func main() {
    scanner := bufio.NewScanner(os.Stdin)

    var target int
    scanner.Scan()
    line := scanner.Text()

    desc := strings.Fields(line)
    target, _ = strconv.Atoi(desc[0])

    res := sqrt(target)

    fmt.Println(res)
}

```

3 二分查找 ✓

```

package main

import (
    "bufio"
    "os"
    "strings"
    "strconv"
    "fmt"
)

func search(nums []int, target int) int {
    if len(nums) == 0 {

```

```

        return -1
    }

    left, right := 0, len(nums)
    for left <= right {
        mid := (left + right) / 2
        if nums[mid] == target {
            return mid
        } else if nums[mid] > target {
            right = mid - 1
        } else {
            left = mid + 1
        }
    }

    return -1
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    line2, _ := reader.ReadString('\n')

    desc := strings.Fields(line1)
    numsStr := strings.Fields(line2)

    numsLen, _ := strconv.Atoi(desc[0])
    target, _ := strconv.Atoi(desc[1])

    nums := make([]int, numsLen)
    for k, v := range numsStr {
        val, _ := strconv.Atoi(v)
        nums[k] = val
    }

    res := search(nums, target)

    fmt.Println(res)
}

```

寻找两个正序数组的中位数

排序（1）

3 排序数组

```

package main

import (

```

```

"bufio"
"os"
"strings"
"strconv"
"fmt"
"math/rand"
)

func quickSort(nums []int, left, right int) {
    if left >= right {
        return
    }

    // 随机选 pivotIndex
    pivotIndex := left + rand.Intn(right - left + 1)
    nums[pivotIndex], nums[right] = nums[right], nums[pivotIndex]

    pivot := nums[right]

    // 分区
    i := left
    for j := left; j < right; j++ {
        if nums[j] < pivot {
            nums[i], nums[j] = nums[j], nums[i]
            i++
        }
    }

    nums[i], nums[right] = nums[right], nums[i]

    // 递归左右
    quickSort(nums, left, i-1)
    quickSort(nums, i+1, right)
}

func main() {
    reader := bufio.NewReader(os.Stdin)

    line1, _ := reader.ReadString('\n')
    line2, _ := reader.ReadString('\n')

    desc := strings.Fields(line1)
    numsLen, _ := strconv.Atoi(desc[0])

    numsStr := strings.Fields(line2)
    nums := make([]int, numsLen)
    for k, v := range numsStr {
        val, _ := strconv.Atoi(v)
        nums[k] = val
    }

    quickSort(nums, 0, numsLen-1)
}

```

```
// newNumsStr := make([]string, numsLen)
// for k, v := range nums {
//     val := strconv.Itoa(v)
//     newNumsStr[k] = val
// }

// fmt.Println(strings.Join(newNumsStr, " "))

for k, v := range nums {
    fmt.Print(v)
    if k != numsLen-1 {
        fmt.Print(" ")
    }
}
}
```